

Face Recognition Docker

Harry Potter Character Recognizer

Projet	4IABD — Application de reconnaissance faciale
Branche Git	structure-docker
Dépôt	https://github.com/pau-anto/Face-recognition-docker
Date	Juin 2026
Technologies	Python · Streamlit · Docker · MySQL · FastAPI

1. Introduction

Ce projet, développé dans le cadre d'une formation 4IABD, propose une application web complète de reconnaissance faciale des personnages de la saga *Harry Potter*. L'objectif pédagogique est de mettre en œuvre une architecture microservices conteneurisée avec Docker, intégrant un frontend interactif, une API de machine learning et une base de données relationnelle.

L'utilisateur peut uploader une photo d'un personnage. L'application l'envoie à une API de reconnaissance faciale qui retourne le nom du personnage identifié ainsi qu'un score de confiance. Les résultats sont persistés en base de données MySQL pour analyse ultérieure.

2. Architecture du projet

Le projet suit une architecture microservices en 3 couches, chaque service étant isolé dans son propre conteneur Docker et communiquant via un réseau Docker interne :

Service	Technologie	Port exposé	Rôle
frontend	Streamlit + Python 3.11	8501 (public)	Interface utilisateur web
api	FastAPI + Python	8000 (interne)	Service de reconnaissance faciale
database	MySQL	3306 (interne)	Persistence des prédictions

2.1 Schéma de communication

Le flux de données suit un chemin unidirectionnel : l'utilisateur interagit uniquement avec le frontend (port 8501). Le frontend communique avec l'API via l'URL interne **http://api:8000** (résolu par le DNS Docker). L'API, après traitement, peut écrire les résultats dans la base de données MySQL.

3. Module Frontend

3.1 Interface utilisateur (app.py)

Le frontend est développé avec **Streamlit 1.32.0**, framework Python spécialisé dans les applications de données. L'interface propose un layout en deux colonnes :

- **Colonne gauche** : zone d'upload d'image (JPG/PNG) avec prévisualisation
- **Colonne droite** : affichage du résultat (personnage, score de confiance)
- **Footer** : indicateur de statut de l'API (■ connectée / ■ non joignable)
- **Badges de maison** : Gryffondor, Serpentard, Serdaigle, Poufsouffle avec couleurs dédiées

3.2 Appel API

L'image uploadée est transmise en multipart/form-data via une requête POST à l'endpoint **/analyze/**. Un health-check préalable sur **/health** permet de désactiver le bouton si l'API est indisponible. La gestion des erreurs couvre les cas de connexion échouée, timeout et erreurs HTTP.

Extrait du code d'appel :

```
response = requests.post( f"{API_URL}{ENDPOINT}", files={"file":
(uploaded_file.name, uploaded_file, uploaded_file.type)}, timeout=15 )
```

3.3 Dockerfile du frontend

Le service frontend utilise une image légère **python:3.11-slim**. Le serveur Streamlit écoute sur **0.0.0.0:8501** pour être accessible depuis l'extérieur du conteneur :

```
FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install
--no-cache-dir -r requirements.txt COPY . . EXPOSE 8501 CMD ["streamlit",
"run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

3.4 Dépendances

Bibliothèque	Version	Utilisation
streamlit	1.32.0	Framework d'interface web
requests	2.31.0	Appels HTTP vers l'API
Pillow	10.2.0	Chargement et affichage des images

4. Module Base de données

La persistance des données est assurée par un serveur **MySQL** initialisé au démarrage via le script **init.sql**. La base de données est accessible uniquement en interne (non exposée à l'extérieur du réseau Docker).

4.1 Schéma de la base de données

```
CREATE DATABASE IF NOT EXISTS hpdb; USE hpdb; CREATE TABLE IF NOT EXISTS
predictions ( id INT AUTO_INCREMENT PRIMARY KEY, character_name VARCHAR(100)
NOT NULL, confidence FLOAT NOT NULL, created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP );
```

La table **predictions** enregistre chaque reconnaissance effectuée : nom du personnage identifié, score de confiance (float entre 0 et 1) et horodatage automatique.

Colonne	Type	Contrainte	Description
id	INT	PK, AUTO_INCREMENT	Identifiant unique
character_name	VARCHAR(100)	NOT NULL	Nom du personnage reconnu
confidence	FLOAT	NOT NULL	Score de confiance (0.0 → 1.0)
created_at	TIMESTAMP	DEFAULT NOW()	Date/heure de la prédiction

5. Déploiement Docker

Le projet utilise **Docker Compose** pour orchestrer les 3 services. Cette approche garantit la reproductibilité de l'environnement et simplifie le démarrage complet de l'application en une seule commande.

5.1 Commandes de démarrage

```
# Construire et lancer tous les services docker-compose up --build # Lancer
en arrière-plan docker-compose up -d --build # Arrêter les services
docker-compose down
```

5.2 Accès aux services

Service	URL d'accès	Visibilité
Frontend Streamlit	http://localhost:8501	Public (navigateur)
API REST	http://api:8000	Interne Docker uniquement
Base de données	hpdb @ port 3306	Interne Docker uniquement

5.3 Variables d'environnement

Le frontend accepte la variable d'environnement **API_URL** pour configurer l'URL de l'API. Par défaut, elle pointe vers **http://api:8000** (nom du service Docker). Pour un déploiement en production, cette variable permet de cibler n'importe quelle URL d'API.

6. Tests

Le projet intègre une structure de tests unitaires répartie dans deux dossiers dédiés au sein de chaque service :

- **frontend/tests/test_app.py** — Tests de l'application Streamlit
- **database/tests/test_schema.sql** — Tests de validation du schéma SQL

Les fichiers de tests sont actuellement présents en structure mais restent à compléter avec des cas de test concrets (mocks de l'API, validation des réponses, tests d'intégration Docker).

6.1 Lancer les tests

```
# Depuis le répertoire frontend pip install pytest pytest frontend/tests/
```

7. Limites et améliorations possibles

Limites identifiées :

- Le service API (modèle de reconnaissance faciale) n'est pas inclus dans la branche *structure-docker*

- Les tests unitaires sont vides — ils doivent être implémentés
- Aucun mécanisme d'authentification ou de rate-limiting sur l'API
- La base de données ne persiste pas les données entre redémarrages (volume Docker non configuré)

Améliorations suggérées :

- Ajouter un volume Docker pour la persistance MySQL
- Implémenter des tests d'intégration avec **httpx** ou **pytest**
- Ajouter un reverse proxy Nginx devant Streamlit
- Configurer des health checks Docker natifs dans docker-compose.yml
- Mettre en place un pipeline CI/CD (GitHub Actions)
- Ajouter un historique des prédictions visible dans l'interface

8. Conclusion

Le projet **Face Recognition Docker** constitue une base solide pour une application de reconnaissance faciale containerisée. La séparation en microservices (frontend, API, base de données) respecte les bonnes pratiques de développement moderne et facilite la maintenance, le déploiement et la scalabilité.

La branche *structure-docker* met en place l'ossature complète du projet : interface utilisateur fonctionnelle, schéma de base de données, Dockerfiles et structure de tests. L'intégration du modèle de reconnaissance faciale (service API) constitue l'étape suivante naturelle pour rendre l'application pleinement opérationnelle.